

# **School Of Computer Science, UPES**

## **Deep Learning: Project Report**

**BTech 5<sup>th</sup> Semester**

**2024-2025**

**Project Title:** Gesture-Controlled Snake Game using MediaPipe

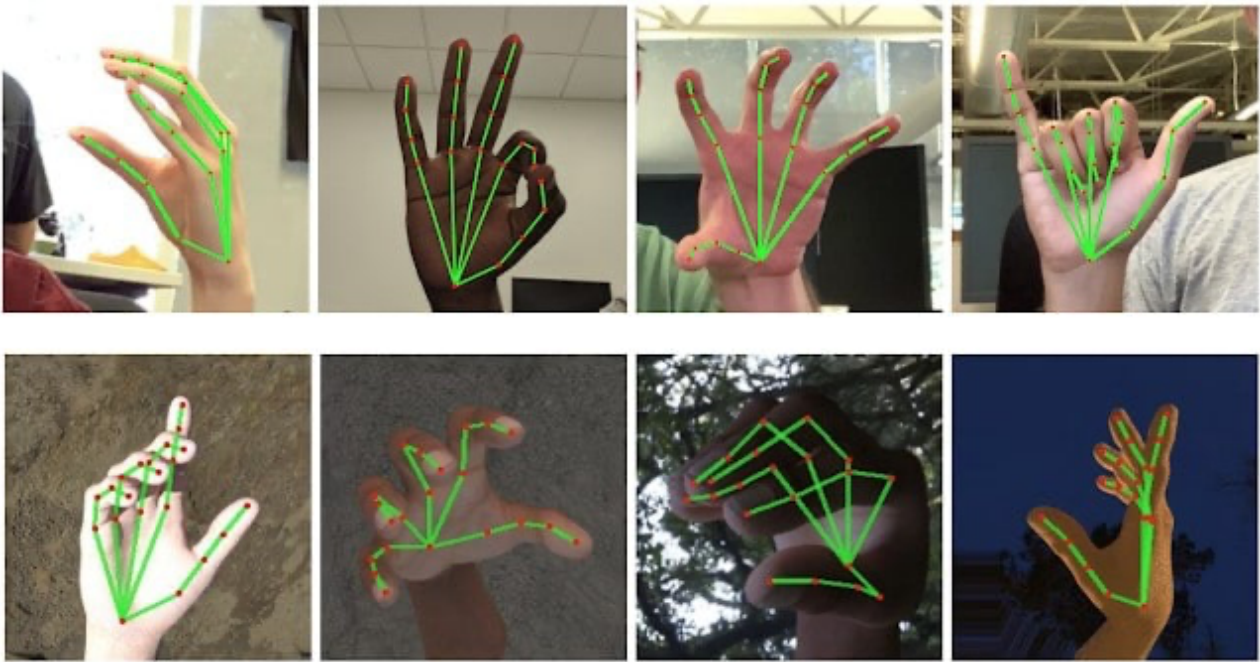


**Submitted To:** Dr. Santosh Panda

**Submitted By:**

1. Ayush Gupta  
SapId: 500125537
2. Arsh Gupta  
SapId: 500122397
3. Anshika Tomar  
SapId: 500119658
4. Bhavya Mittal  
SapId: 500119005

## Abstract



Human–computer interaction (HCI) has evolved significantly over the last decade, moving from traditional input devices such as keyboards and mice to more intuitive and natural interaction methods. One emerging direction in this field is vision-based gesture recognition, where deep-learning-powered models interpret human hand movements directly from a video feed. Such systems eliminate the need for physical controllers and enable seamless, contact-free interaction, making them highly suitable for gaming, virtual reality, assistive technologies, and smart environments. With advancements in deep learning and computer vision, libraries like MediaPipe have made real-time hand-tracking highly efficient and accessible. MediaPipe’s Hand Landmark Model uses convolutional neural networks to detect 21 keypoints of the human hand with high precision, enabling gesture recognition from simple webcam input. This project leverages these capabilities to design an interactive system where users can control the classic Nokia Snake Game using only their hand movements.

The aim of this project is to develop a gesture-driven control interface that maps the index fingertip motion to the directional movements of the snake. The system detects finger position, tracks motion vectors, and converts them into UP, DOWN, LEFT, and RIGHT commands. Additionally, open-palm detection is used to provide simple control actions like game restart. The integration of deep learning–based hand tracking with a real-time game environment demonstrates how modern AI techniques can transform traditional applications into immersive experiences. This project reflects key concepts taught in the Deep Learning curriculum—such as convolutional neural networks, landmark detection, and real-time inference—while also showcasing their practical use in building interactive systems. By combining MediaPipe-based gesture recognition with Pygame for game rendering, this system highlights how deep learning can be applied beyond classification tasks to create intuitive, human-centered interfaces.

## **Acknowledgement**

We would like to express our sincere gratitude to Dr. Santosh Panda for his invaluable guidance, encouragement, and continuous support throughout the development of this project. His expert insights, patience, and commitment to teaching have played a crucial role in strengthening our understanding of Deep Learning concepts and their real-world applications. Under his mentorship, we were able to explore advanced ideas in computer vision, gesture recognition, and deep-learning-based landmark detection, which significantly shaped the design and implementation of this project.

We are deeply appreciative of his constructive feedback, which helped us refine our methodology, improve our system's efficiency, and build a more robust and interactive application. His support and motivation throughout the duration of this work have been deeply inspiring.

We also extend our heartfelt thanks to all the faculty members of our department for providing a supportive and academically enriching environment. Their mentorship and encouragement have played a vital role in our growth as Computer Science engineering students.

Finally, we acknowledge the teamwork, dedication, and collaborative effort of every member involved in this project. Countless discussions, planning sessions, and problem-solving moments enabled us to successfully complete this work.

Team Members:

- Bhavya Mittal
- Arsh Gupta
- Ayush Gupta
- Anshika Tomar

# Literature Review

## I. Introduction to Gesture Recognition in Human-Computer Recognition

Human–computer interaction (HCI) has evolved from simple command-line interfaces to modern, highly intuitive interfaces that rely on speech, touch, and gestures. Among these, gesture recognition plays an increasingly important role due to its ability to provide a natural, hands-free method of communication between humans and machines. Gestures are inherently intuitive, universally understood, and do not require any physical device, making them ideal for interactive applications such as gaming, virtual reality, robotics, and assistive technologies.

The earliest gesture-recognition systems relied on manually designed computer vision algorithms such as colour segmentation, edge detection, and motion tracking. Although these approaches laid the foundation for gesture-based interfaces, they suffered from severe limitations including sensitivity to lighting changes, background clutter, inconsistent skin detection, and inability to handle complex hand shapes. These issues made such systems unreliable for real-world use.

The introduction of deep learning, particularly convolutional neural networks (CNNs), transformed gesture recognition by enabling models to learn features automatically from large datasets. Deep learning–driven hand tracking and pose estimation have now become standard in modern HCI systems. This shift has enabled robust, real-time gesture detection even under challenging conditions such as varied lighting, motion blur, or partial occlusion. This project—**Gesture-Controlled Snake Game using MediaPipe**—builds directly on this evolution by using a state-of-the-art deep learning hand-tracking model to create an interactive gaming experience controlled entirely through hand movements.

## II. Deep Learning and Hand Pose Estimation

### 1. Convolutional Neural Networks (CNNs) for Vision Tasks

With the rise of affordable GPUs and massive annotated datasets, deep learning has made high-precision hand landmark detection accessible even on personal devices. At the heart of this progress is the Convolutional Neural Network (CNN). Unlike traditional approaches that required manual feature engineering (such as SIFT, Haar cascades, or HOG), CNNs excel at learning representations automatically. This makes them ideal for interpreting the complex spatial data of human hands. Consequently, the industry has moved on from older techniques, with CNNs now powering the most popular tracking tools like DenseHand and OpenPose. Their ability to handle the diverse motions and geometries of hands has cemented their place as the superior choice for detection and segmentation tasks. As these models became more efficient, researchers began adapting them for real-time applications, enabling continuous tracking without sacrificing accuracy. This shift has transformed hand-gesture interfaces from experimental concepts into reliable components of everyday interactive systems.

## **2. Evolution of Hand Pose Estimation**

Hand pose estimation refers to predicting the precise locations of hand joints, typically represented as 2D or 3D coordinates. Early works in this domain employed template matching or 2D contour-based methods, which failed under complex poses or occlusions. Later studies such as OpenPose introduced multi-stage CNN architectures capable of predicting body, face, and hand landmarks using part affinity fields. Although accurate, such models were computationally expensive and unsuitable for real-time applications without dedicated GPU hardware. Subsequent advancements introduced heatmap regression, where CNNs predicted probability distributions for keypoint positions. Datasets like RHD (Rendered Hand Dataset), CMU Panoptic Hand Dataset, and FreiHAND enabled high-quality supervised training.

However, limitations persisted:

- High computational load
- Difficulty running on CPU
- Inefficient for mobile or embedded platforms
- Sensitivity to occlusions

These limitations made hand pose estimation challenging for real-time gaming applications—until the introduction of MediaPipe Hands.

## **III. MediaPipe Hands: A Lightweight, Real-Time Model**

Developed by Google Research, MediaPipe Hands represents a breakthrough in real-time hand landmark detection. Its architecture is specifically optimised for speed, robustness, and cross-platform use. MediaPipe Hands uses a two-stage deep learning pipeline:

### **Stage 1: Palm Detection**

Rather than detecting the entire hand (which has a highly variable shape), MediaPipe uses palm detection modelled using a specialised Single Shot Detector (SSD). Palms exhibit limited variation in shape, making detection far more stable than full-hand detection. This approach provides:

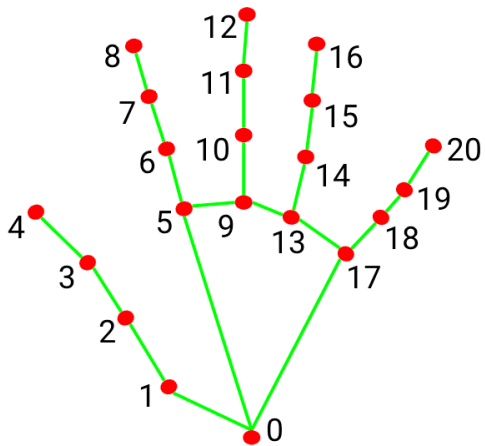
- Higher detection accuracy
- Lower false positives
- Greater robustness under occlusions

The palm detector outputs a bounding box, which is used for the next stage. This initial bounding box gives the system a reliable starting point, even when the fingers are curled or partly hidden. By narrowing the focus to the palm first, the model avoids a lot of the confusion that normally arises with full-hand shapes. This clean crop then becomes the input for the second stage, where the finer joint positions are predicted.

## Stage 2: Hand Landmark Model (21-Point Regression)

Once the palm region is isolated, a second neural network performs **direct regression** to output 21 3D hand landmarks. These points include:

- 5 fingertips
- 4 mid-finger joints
- 5 proximal finger joints
- Wrist joint

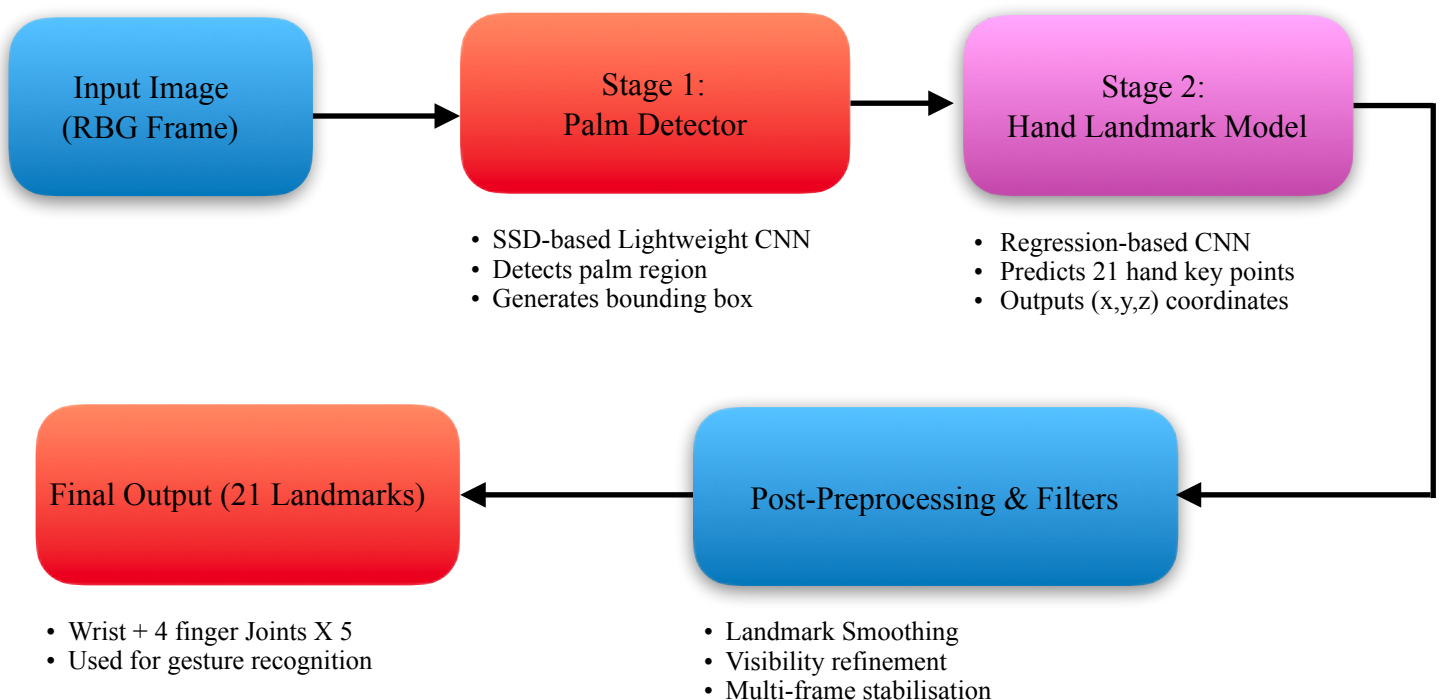


- |                       |                       |
|-----------------------|-----------------------|
| 0. WRIST              | 11. MIDDLE_FINGER_DIP |
| 1. THUMB_CMC          | 12. MIDDLE_FINGER_TIP |
| 2. THUMB_MCP          | 13. RING_FINGER_MCP   |
| 3. THUMB_IP           | 14. RING_FINGER_PIP   |
| 4. THUMB_TIP          | 15. RING_FINGER_DIP   |
| 5. INDEX_FINGER_MCP   | 16. RING_FINGER_TIP   |
| 6. INDEX_FINGER_PIP   | 17. PINKY_MCP         |
| 7. INDEX_FINGER_DIP   | 18. PINKY_PIP         |
| 8. INDEX_FINGER_TIP   | 19. PINKY_DIP         |
| 9. MIDDLE_FINGER_MCP  | 20. PINKY_TIP         |
| 10. MIDDLE_FINGER_PIP |                       |

Each landmark has x, y (normalised image coordinates) and z (relative depth). The lightweight nature of the model allows inference at real-time speeds (20–30 FPS) even on CPU.

This architecture is ideal for the project because it offers:

- High precision for fingertip tracking
- Low latency
- CPU-friendly inference
- Consistent performance under real-world conditions



## IV. Gesture Recognition Technique and Related Work

Gesture recognition has been widely explored within the fields of computer vision and human–computer interaction. Over the years, researchers have proposed different methods for understanding hand movements, each with its own strengths and limitations. Broadly, gesture-recognition strategies can be divided into three major categories: static gesture recognition, dynamic gesture recognition, and keypoint-based gesture analysis. These approaches form the foundation for modern systems like MediaPipe Hands, and they also directly influence how gesture-driven applications—such as this Snake Game—are designed.

### 1. Static Gesture Recognition

Static gesture recognition classifies fixed hand poses from a single frame, such as an open palm, closed fist, "OK" sign, thumbs-up, or number gestures. Previous approaches to hand-gesture recognition have traditionally depended on handcrafted features such as skin-color segmentation, contour matching, or template comparison. However, these conventionally used techniques had limited performance in view of changes in lighting, skin tone variation, and complex backgrounds.

With the introduction of deep learning, CNN-based classifiers began to dominate in the static gesture recognition arena. Nowadays, it is possible for models to learn texture, shape, and orientation of hands directly from data, significantly increasing robustness. From the perspective of this project, static recognition is utilised for detecting an open palm—a very simple and intuitive gesture for actions such as starting or restarting the game. This tends to align with established research showing that static poses are reliable for "command-type" interactions where the user needs a clear, intentional trigger gesture.

### 2. Dynamic Gesture Recognition

Dynamic gesture recognition interprets motion across several frames, thus essential for understanding actions like swiping, waving, pushing, or drawing. Early systems for dynamic gestures tracked along paths of motion by using optical flow, motion templates, or background subtraction. Although these worked in controlled environments, these were highly sensitive to noise and camera instability. Deep learning expanded this field significantly. The models, including RNNs, LSTMs, and 3D CNNs, learned the temporal patterns to recognize complex motions quite precisely. However, these models require large labeled datasets and substantial computational power. For lightweight real-time applications, many systems, including this project, would prefer a simpler and more efficient technique: motion-vector-based analysis. Instead of classifying full gesture sequences, the system computes:

- **movement = current\_position – previous position**

This methodology is further backed up by numerous studies that prove that small motions, especially those of the fingers, can be reliably mapped to intentional user directions. These motion vectors will form the backbone of the snake movement logic in this project, telling whether the user is moving their finger upward, downward, leftward, or rightward.

### 3. Key point-Based Gesture Recognition

Key point-based gesture recognition has become the dominant method for modern systems thanks to advances in pose estimation. Instead of analysing entire images, this approach studies the **precise coordinates of anatomical landmarks** such as joints, fingertips, and knuckles. Once these points are identified, the gestures can be interpreted using geometric relationships, angles, or trajectories.

MediaPipe Hands adopts this strategy by predicting **21 hand landmarks** using a deep regression network. Each landmark includes an (x, y) coordinate and a relative depth value (z). This structured representation opens the door to a wide range of gesture-interpretation methods:

- Fingertip trajectory for directional control
- Distance checks for pinch detection
- Finger-joint angles for pose recognition
- Hand orientation for navigation tasks

This project uses this approach by focusing on a single landmark—the **index fingertip (Landmark 8)**. Its movement between consecutive frames forms a stable and intuitive signal for determining the direction of gameplay. Research consistently shows that tracking a single reliable keypoint is often more effective for real-time interactions than interpreting full hand poses, especially in low-latency applications such as gaming.

### 4. Gesture-Based Interfaces in Interactive Systems

A large amount of literature covers the use of gesture-controlled interfaces with regards to game interfaces and virtual environments. Systems using devices like Microsoft Kinect, Leap Motion, and different AR toolkits have demonstrated that tracking the body and hands can be highly immersive. Most of these technologies, however, rely on specialised sensors or depth cameras, limiting their accessibility and cost-effectiveness.

Recent studies show that there is a strong trend toward camera-only solutions driven by deep-learning-based landmark detectors. Applications span virtual drawing tools, sign-language translators, robot control, and air-cursor systems. These works share the same challenge: finding a good trade-off between accuracy and computational efficiency to grant real-time responsiveness.

This project further contributes to the evolving landscape by showing that such a classic game can be controlled seamlessly with just a webcam and CPU, thanks to MediaPipe's optimized architecture. This is a concrete example of how gesture-based interaction can be integrated into a traditional game engine such as Pygame-acting like a bridge between deep-learning research and practical applications.



# Methodology

This project combines deep learning–based hand landmark detection with a traditional 2D game engine to create a fully gesture-controlled version of the classic Snake game. The methodology consists of multiple interconnected stages: capturing input from the webcam, processing the frames through MediaPipe’s hand-tracking pipeline, extracting and smoothing keypoint data, interpreting gesture directions, and finally translating those gestures into movement commands within the game. This section explains each stage in detail and presents the technical workflow followed throughout development.

## 1. System Architecture Overview

The architecture of this project is built around two main subsystems that work together to enable gesture-based control of the Snake game. Each subsystem handles a specific part of the workflow, and both operate simultaneously to maintain real-time performance.

### A. Gesture Preprocessing Subsystem (Deep Learning + Computer Vision)

This subsystem is responsible for interpreting the user’s hand movements and transforming them into meaningful control signals for the game. It consists of several key components:

#### 1. Video Frame Acquisition

A continuous stream of frames is captured from the webcam using OpenCV. Each frame is mirrored (flipped horizontally) to make the gesture interaction feel natural and intuitive—when the player moves their hand to the right, the snake also moves to the right.

#### 2. Frame Pre-Processing

Since MediaPipe requires RGB format, each incoming frame is converted from OpenCV’s default BGR format to RGB. This step ensures compatibility with the deep-learning model.

#### 3. MediaPipe Hand Landmark Extraction

The processed frame is passed to the **MediaPipe Hands** model, which performs:

- Palm detection
- Cropped hand extraction
- 21-landmark regression

Among these 21 keypoints, Landmark **8** (the index fingertip) is the most critical for this project. Its movement across frames is used to infer direction (UP, DOWN, LEFT, RIGHT).

#### 4. Gesture Interpretation Logic

Once the fingertip position is extracted, the subsystem performs:

- Position buffering
- Smoothing (to reduce jitter)
- Motion vector calculation

- Threshold filtering
- Majority voting
- Cooldown regulation

This pipeline transforms raw fingertip motion into a **stable and reliable direction command**.

To prevent lag in the main game, this entire gesture-processing subsystem runs inside a **background thread**.

## **B. Game Control Subsystem (Pygame Engine)**

This subsystem handles the actual execution of the Snake game and responds to the directional gestures received from the gesture processor.

### **1. Direction Input Handling**

The Pygame engine reads the output of the gesture processing thread through shared variables such as `current_gesture`. This is equivalent to receiving an arrow-key input but generated through hand movement.

### **2. Game Logic Updating**

The game continuously updates internal logic, including:

- Snake movement
- Snake growth upon eating food
- Randomized fruit spawning
- Score calculation
- Wrap-around world boundaries
- Self-collision detection
- Game-over state transitions

### **3. Rendering and User Interface**

Pygame draws:

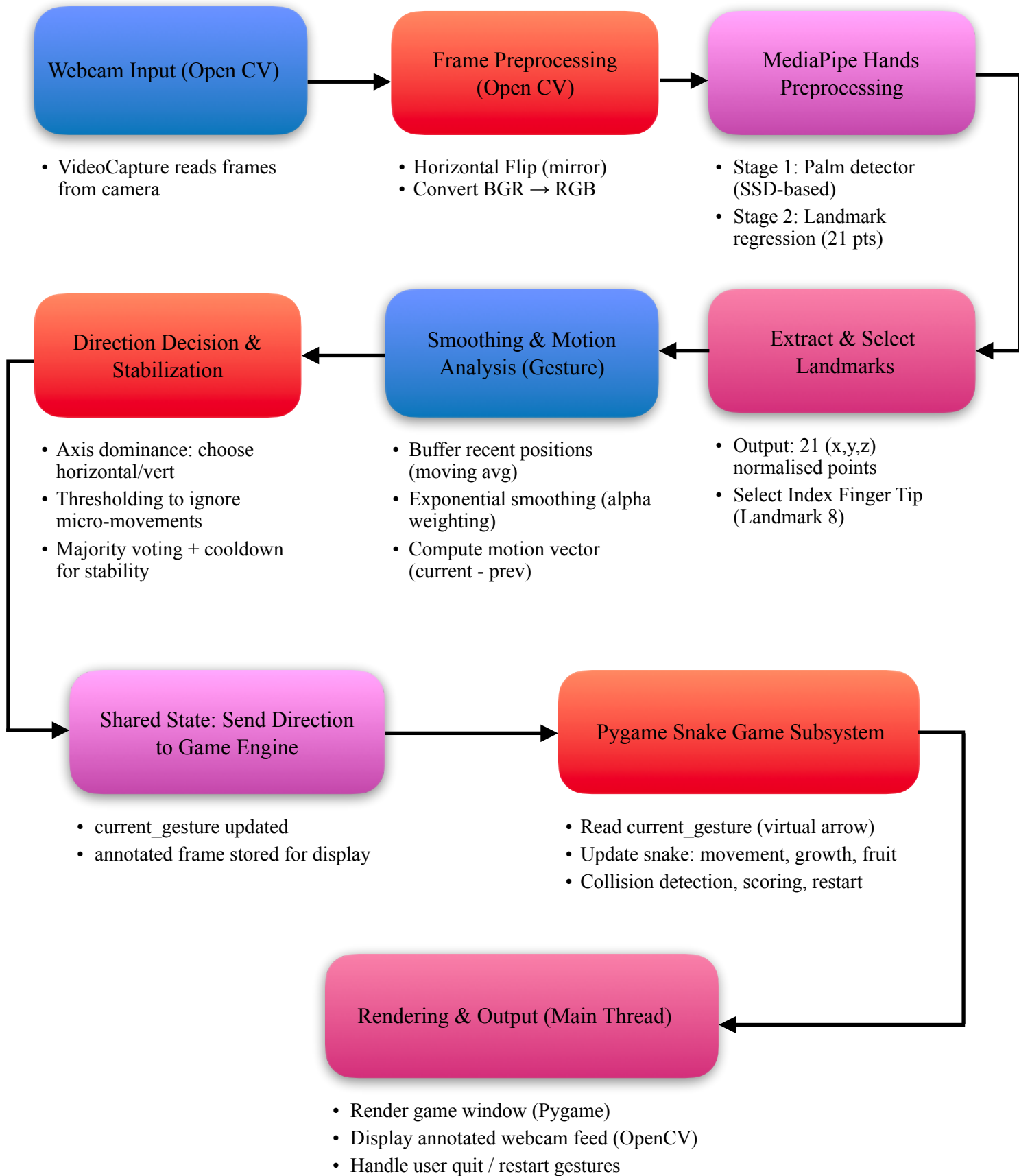
- The snake body
- The fruit object
- Particle effects
- The grid background
- Scoreboard and “GAME OVER” screens

This rendering occurs in the main thread to avoid issues with GUI responsiveness.

### **4. Restart Logic**

Using an open-palm gesture, the game can be restarted without any keyboard input—making the entire gameplay contact-free.

## Project Pipeline



## II. Detailed Workflow

This subsection explains the runtime flow of the system in detail and points to the specific files and functions that implement each step.

### 1. Program StartProgram start (main entrypoint)

- File: main.py
- Function: main() → constructs GameManager() and calls game\_manager.run().
- Role: initialises the game and the gesture subsystem and starts threads.

### 2. Camera initialisation & capture

- File: main.py
- Function: GameManager.initialize\_camera()
- Role: opens the webcam (cv2.VideoCapture(0)) and sets resolution (640×480). If the camera cannot be opened, the program exits gracefully.

### 3. Gesture detection thread

- File: main.py
- Function: GameManager.gesture\_detection\_loop() (runs in a background thread)
- Role: repeatedly reads frames from the camera (cap.read()), flips them horizontally for natural interaction, and sends each frame to the gesture module (self.gesture\_controller.detect\_gestures(frame)).

### 4. Gesture processing

- File: gesture\_controller.py
- Function: GestureController.detect\_gestures(frame)
- Role: runs MediaPipe Hands on the RGB-converted frame, extracts landmarks, performs fingertip smoothing and motion analysis, detects open-palm and other gestures, then returns (direction, is\_pinching, annotated\_frame).

### 5. Shared state update

- File: main.py
- Code path: inside gesture\_detection\_loop()
- Role: the gesture thread writes three shared variables:
  - self.current\_gesture ← direction string ("UP", "DOWN", "LEFT", "RIGHT")
  - self.is\_speed\_boost ← pinch boolean (unused currently)
  - self.latest\_frame ← annotated image (for display)

- Note: simple attribute assignment is used for communication; this is acceptable for small projects though a Queue or Lock can be used for strict thread-safety.

## **6. Main loop: reading gestures and running the game**

- File: main.py
- Function: GameManager.run()
- Role: main thread displays the annotated webcam frame (cv2.imshow()), listens for keyboard/quit events, reads self.current\_gesture and calls self.game.change\_direction(self.current\_gesture), calls self.game.update() on a timed cadence, and calls self.game.draw() to render the state.

## **7. Game logic & rendering**

- File: snake\_game.py
- Key functions:
  - SnakeGame.update() — advances the snake, handles collisions, fruit eating, particle effects.
  - SnakeGame.change\_direction(new\_direction) — maps gesture string to Direction enum and updates next\_direction if not reversal.
  - SnakeGame.draw() — draws board, snake segments, fruit, particles and UI (score, game over).
- Role: all game mechanics run here; gestures only update next\_direction.

## **8. Restart & cleanup**

- When game\_over is True, main.py checks gestures and calls self.game.handle\_restart(self.current\_gesture) to reset if user shows the restart gesture (open palm or UP depending on your code).
- On exit, GameManager.cleanup() releases the camera, destroys OpenCV windows, quits Pygame, and joins the gesture thread.

### III. Gesture Detection Logic

This section isolates the gesture-detection algorithm itself. It explains every processing step from a raw frame to the final stable direction.

#### A. Input

- A single webcam frame (BGR) captured by OpenCV and horizontally flipped.

#### B. Pre-processing

- Convert BGR → RGB (MediaPipe expects RGB).
- Optionally, resize or normalise images (your code uses normalised coordinates from MediaPipe).

#### C. MediaPipe Hands Inference

- Stage 1: Palm detector finds the palm bounding box.
- Stage 2: Landmark regression returns 21 normalized (x, y, z) points.
- Output: `hand_results.multi_hand_landmarks` contains one or more `hand_landmarks`.

#### D. Landmark selection

- Index fingertip is extracted:
- `index_tip = hand_landmarks.landmark[self.mp_hands.HandLandmark.INDEX_FINGER_TIP]`
- `current_pos = np.array([index_tip.x, index_tip.y])`
  - Coordinates are normalized between 0 and 1.

#### E. Buffer & Smoothing

- Append `current_pos` to a bounded deque (`pos_buffer`, `maxlen=4`).
- Compute `avg_pos = mean(pos_buffer)`.
- Exponential smoothing:  
$$\text{smoothed\_pos} = \alpha * \text{avg\_pos} + (1 - \alpha) * \text{smoothed\_pos\_prev}$$

where  $\alpha = 0.65$  in your code.

#### F. Motion Vector Computation

- If `previous_position` exists:  
$$\text{movement} = \text{smoothed\_pos} - \text{previous\_position}$$
- Compute `magnitude = np.linalg.norm(movement)`.

#### G. Thresholding

- Minimum movement threshold: `gesture_threshold` (0.03 in your code).
- If `magnitude <= threshold` → ignore (no direction change).

#### H. Axis Dominance

- Compare absolute components:
  - If `abs(movement[0]) > abs(movement[1])` → horizontal candidate:
    - `movement[0] > 0` → RIGHT

- else → LEFT
- Else → vertical candidate:
  - movement[1] > 0 → DOWN
  - else → UP

### **I. Stabilization**

- Append candidate to `direction_buffer` (deque maxlen=3).
- Use majority vote (Counter) — require frequency  $\geq 2$  to accept.
- If accepted and differs from `current_direction`, set `current_direction` = most\_common and set `gesture_cooldown` = max\_cooldown.
- Cooldown decrements each frame to avoid rapid alternation.

### **J. Open-Palm Detection**

- Count fingers using landmark geometry (`fingers_up()` checks tips vs pip positions).
- If fingers  $\geq 4$  across a palm buffer majority → set `is_open_palm` = True (used for restart).

### **K. Output**

- Return: (`self.current_direction`, `is_pinching`, `annotated_frame`)

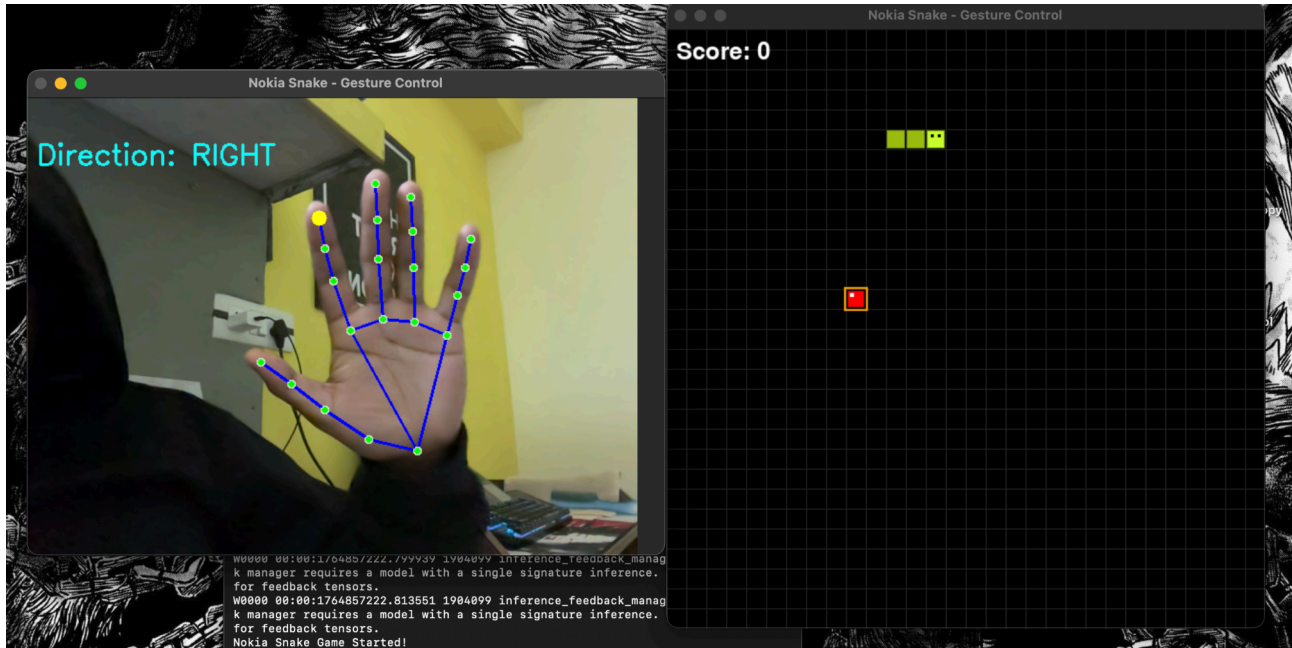
## **Interpretation:**

To detect gestures, each webcam frame is first converted into a format that MediaPipe can understand, ensuring the deep-learning model receives clean and consistent input. MediaPipe then identifies the presence of a hand and predicts 21 precise landmark points across the fingers and palm, including the tip of the index finger, which acts as the primary control point in this system. The algorithm focuses on the index fingertip because its position naturally reflects the user's intended direction without requiring any special posture or effort. As the user moves their finger, the system tracks its motion across multiple consecutive frames and applies smoothing techniques to reduce noise, small jitters, or accidental micro-movements caused by hand tremors or camera fluctuations.

Once the fingertip's position is stabilized, the system compares its current location with its previous location to determine how the finger is moving—upward, downward, leftward, or rightward. A minimum movement threshold is used so that very small or unintended motions do not trigger false direction changes. To further improve reliability, the detected direction must appear consistently over a short sequence of frames before being accepted as a valid command. This prevents rapid switching and ensures that direction changes feel natural and controlled during gameplay. Additionally, the system analyzes how many fingers are extended; if four or more fingers are detected, the gesture is interpreted as an open palm, which is used as a simple and intuitive restart gesture. The system then sends the final direction to the game engine, which updates the snake's movement instantly.

## Observation and Results

The Snake game, controlled by gestures, showed smooth and reliable performance under normal indoor lighting conditions during tests. The MediaPipe hand-tracking module detected the index fingertip every time and translated its movements into accurate directional commands. Small hand jitters were filtered out thanks to the smoothing and thresholding logic, which allowed for stable and easy control of the game. Users felt that moving their finger in four directions resulted in predictable and prompt snake movements, with almost no visible delay. Also, the open-palm gesture for restarting the game was detected reliably, given that the hand was clearly visible to the camera.



Overall, the system provided a responsive and intuitive user experience. The game kept running at a high frame rate, while gesture recognition was preserved even when users were at different distances from the webcam. Only low-light conditions slightly decreased the detection precision, which is expected for most vision-based systems. Users reported a high level of engagement and fun, highlighting that the gestures brought a whole new feeling to the modern and interactive Snake game. These observations confirm that landmark detection using MediaPipe in combination with the custom gesture logic can be effectively used to implement real-time hands-free control of games on standard hardware.

## Future Scope

This project opens several directions for future enhancement, particularly in the domain of deep-learning-based gesture understanding. The current system uses MediaPipe's pretrained CNN, but future versions could incorporate custom-trained models optimized for specific gestures or multi-hand interactions. Adding recurrent architectures or temporal models—such as LSTM-based gesture classifiers or transformer-based motion encoders—could further improve temporal stability and enable recognition of more complex dynamic gestures.

Beyond gaming, the underlying deep-learning pipeline used in this project can be extended to a wide range of real-world applications. Because MediaPipe provides robust, model-derived spatial keypoints, the same architecture can support assistive technologies for individuals with limited



mobility, enabling hands-free computer navigation or smart-home control. Gesture-based interfaces are also becoming central to AR/VR systems, where intuitive, camera-only interaction reduces dependence on physical controllers. In robotics, fingertip and hand-pose tracking can be used to guide manipulators, trigger commands, or perform teleoperation tasks, creating a natural communication channel between humans and machines. These extensions highlight the versatility of CNN-based landmark extractors as general-purpose perception modules.

Future enhancements could incorporate additional deep-learning components that further improve system intelligence and adaptability. Integrating depth estimation networks or 3D hand-pose regression models would provide richer geometric information, enabling more precise gesture recognition even under occlusion. Reinforcement learning agents could be introduced to personalize gesture thresholds or adapt motion sensitivity based on user behavior over time.

## **Conclusion**

This project demonstrates how deep learning can be used to create real-time, intelligent human–computer interaction systems using only consumer-grade hardware. Central to the system is MediaPipe’s dual-stage CNN architecture: a palm-detection network performs anchor-based localisation of a hand, and a regression network predicts 21 high-precision hand landmarks. These models execute on a standard CPU; their lightweight design and optimized graph execution serve to illustrate the practical maturity of deep-learning inference pipelines. By incorporating these models with a custom smoothing and gesture-interpretation layer, this project demonstrates how deep-learning representations can be converted into stable, high-level control signals appropriate for interactive applications.

This work, from a methodological point of view, emphasizes some of the most salient deep-learning principles taught in the course: landmark regression, model generalization under variable lighting conditions, multi-stage inference pipelines, and post-processing steps like temporal filtering, thresholding, and stabilization for making robust predictions. The system effectively bridges low-level neural outputs (landmark coordinates) with high-level semantic actions (UP, DOWN, LEFT, RIGHT), thus showing the power of learned representations to enable intuitive and gesture-driven interfaces.

Beyond its immediate application to controlling a Snake game, this project demonstrates the broader possibility of deep-learning-based perception systems in real-time domains such as AR/VR, robotics, assistive technology, and touchless interaction. The results here indicate that carefully engineered deep-learning pipelines can replace traditional hardware sensors with purely vision-based solutions while maintaining robustness and responsiveness. Finally, this project reinforces a key theme of deep learning: models trained on large-scale data can act as general-purpose feature extractors, powering interactive applications that feel natural, adaptive, and human-centered.

## References

1. Bazarevsky, V., Zhang, F., Vakunov, A., Sung, G., Chang, R., & Grundmann, M. (2019). MediaPipe Hands: On-device real-time hand tracking. Google Research.
2. Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools.
3. Cao, Z., Hidalgo, G., Simon, T., Wei, S.-E., & Sheikh, Y. (2021). OpenPose: Realtime multi-person 2D pose estimation using part affinity fields. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(1), 172–186.
4. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
5. Karpathy, A. (2019). CS231n: Convolutional neural networks for visual recognition. Stanford University.
6. Newell, A., Yang, K., & Deng, J. (2016). Stacked hourglass networks for human pose estimation. In *European Conference on Computer Vision (ECCV)* (pp. 483–499). Springer.
7. Pfister, T., Charles, J., & Zisserman, A. (2015). Flowing ConvNets for human pose estimation in videos. In *IEEE International Conference on Computer Vision (ICCV)* (pp. 1913–1921).
8. Shotton, J., Fitzgibbon, A., Cook, M., Sharp, T., Finocchio, M., Moore, R., Kipman, A., & Blake, A. (2011). Real-time human pose recognition in parts from a single depth image. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 1297–1304).
9. Yilmaz, A. E., & Zarb, F. G. (2021). Vision-based hand gesture recognition methods: A review. *IEEE Access*, 9, 107201–107217.
10. Zhang, F., Bazarevsky, V., Vakunov, A., Sung, G., Chang, R., & Grundmann, M. (2020). MediaPipe: A framework for building multimodal ML pipelines. Google Research.